

NanoDB: Design and Implementation of a Custom Relational Database Engine with Constraint Driven Data Structures

Qamar Mehmood

Roll Number: 26I 7800

CS 4002 Applied Programming MS CS Spring 2026

FAST NUCES, Islamabad

<https://github.com/Qamar2001/nanodb>

Abstract—Building a database engine from scratch is one of the most demanding, and most rewarding, ways to understand what actually happens beneath the abstractions that developers take for granted every day. This paper documents the design, implementation, and empirical evaluation of NANODB, a functional mini relational database engine written entirely in C++ with no Standard Template Library (STL) containers. Every core component was implemented by hand: the buffer pool, the LRU eviction cache, the hash map catalog, the AVL index, the SQL expression parser, and the multi table join optimizer. The engine runs on 100,000 real records from the TPC H benchmark [1], evaluates complex WHERE expressions through a Shunting Yard parser operating on a custom stack, and routes multi table joins using a graph based MST algorithm. Empirically, the AVL tree index was roughly 69 times faster than a sequential scan on 20,000 records, and at 100,000 records the speedup reached 879 $\times$. The buffer pool correctly handled 4,950 LRU evictions under a 50 page memory constraint with zero memory errors. This paper details the reasoning behind each design decision, provides mathematical proofs of component efficiency, and presents measured results from the live running system.

Index Terms—relational database, buffer pool, LRU cache, AVL tree, Shunting Yard, query optimization, TPC H benchmark, C++, no STL

I. INTRODUCTION

There is a real difference between knowing how to use a database and understanding how one works. A developer who writes SQL queries every day may never think about what happens inside the engine when that query lands as a raw character string: how the WHERE clause gets broken apart into executable operations, how the planner decides which table to read first, or why a differently ordered join can shift query time from a few milliseconds to several minutes. These are not obscure edge cases. They are the core of what a database engine does, and most programmers never see them directly.

The NANODB project was built to change that, at least personally. The central constraint is simple: no STL. No `std::vector`, no `std::map`, no `std::stack`, no `std::sort`. Every data structure is built from raw arrays and pointers. This restriction is frustrating at first, but it turns out to be an unusually effective way to learn. When you write your own doubly linked list and watch it handle thousands of LRU

evictions in microseconds, while an equivalent array would require millions of element shifts, the performance argument for linked structures stops being something you read about and becomes something you have seen work in practice. The same thing happens with AVL rotations, operator precedence stacks, and heap based scheduling: implementing them yourself, and then watching them run correctly on real data, builds a kind of intuition that is hard to get any other way.

The resulting system is a working relational engine. It reads records from TPC H benchmark files [1] and stores them in a paged binary format on disk. It indexes primary keys using a self balancing AVL tree [2], evaluates SQL WHERE clauses through a Shunting Yard parser on a custom stack, routes multi table joins through a graph based cost optimizer, and handles high priority administrative queries through a binary max heap scheduler. This paper documents how each component was designed, proves its theoretical complexity, and presents empirical benchmarks from the actual running system.

II. RELATED WORK

NANODB draws on several active lines of research in storage systems, indexing, and query optimization. This section situates the design decisions within that broader context, highlighting where existing work directly informed the implementation and where the constraints of a from scratch project led to different choices than a production system might make.

A. Buffer Pool and Memory Management

The idea of managing a fixed pool of memory pages explicitly, rather than letting the OS handle it through memory mapped I/O, has been debated in the systems community for years. Crotty et al. [3] make a strong case for the explicit approach, showing that mmap based engines give up fine grained control over what happens when memory runs short and can suffer from unpredictable latency spikes as a result. Their argument resonated directly with the NANODB design: every page eviction in this engine is a deliberate decision made by the buffer pool, not an opaque OS event.

Neumann and Freitag’s Umbra system [4] takes this further, demonstrating that a carefully engineered disk based engine can reach performance levels close to in memory systems. The key insight from Umbra is that the buffer pool architecture itself is not the bottleneck; what matters is how predictably and efficiently it can be managed. That principle guided the choice of a doubly linked list combined with a hash map for the LRU cache in NANODB, since both eviction and promotion need to be $O(1)$ for the overhead to remain negligible relative to actual disk I/O.

B. Indexing Structures

The AVL tree has a long history in algorithms literature [2] and remains one of the cleaner choices for a primary key index when correctness and predictability matter more than squeezing out extra throughput. The strict height invariant no two subtrees at any node can differ by more than one level keeps lookup time at $O(\log N)$ regardless of insertion order, which is exactly the kind of worst case guarantee that matters for a project where the data comes from a benchmark and insertion order is not under our control.

The learned index line of work, particularly ALEX by Ding et al. [5], is a genuinely interesting alternative. ALEX fits a piecewise linear model to the key distribution and can outperform B tree variants on read heavy workloads with relatively stable data. The tradeoff is complexity: the model needs to be updated as data changes, and its performance degrades if the key distribution shifts in unexpected ways. For NANODB, where inserts and updates happen alongside reads and where the primary goal is to understand the mechanics rather than maximize throughput, the AVL tree’s four well defined rotation cases and straightforward invariant were a better fit.

C. Query Optimization

The join ordering problem is as old as relational databases themselves, and it remains an area of active research. Marcus et al. [6] introduced Bao, a system that uses Thompson sampling to nudge the underlying query optimizer away from historically bad plan choices. Bao does not replace the optimizer entirely but learns, over time, which plan “hints” tend to produce better outcomes on a given workload. The results on benchmarks like JOB and TPC H are substantial.

NANODB’s join optimizer takes a much simpler approach: it models tables as graph nodes, join costs as edge weights, and applies Kruskal’s MST algorithm [2] to find the ordering with the smallest total estimated intermediate result. For two or three tables this works well and can be computed essentially instantly. The more sophisticated machinery of Bao would become relevant once the join set grows large enough that simple cost proxies are no longer reliable, but at the scale of this project the MST approach produces correct orderings in every tested case.

Leis et al. [7] examine a related problem at the scheduling level, showing that the order in which queries are executed in an analytical workload meaningfully affects end to end

throughput and tail latency. NANODB handles this through a binary max heap scheduler, a lightweight but practically important mechanism that ensures administrative operations always preempt background reads.

III. SYSTEM ARCHITECTURE

NANODB is structured as a layered pipeline, where each layer has a single well defined responsibility and communicates with the layers above and below it through narrow interfaces. Fig. ?? gives a conceptual view of the stack.

At the bottom, the **Pager** handles raw binary I/O, reading and writing fixed size pages to disk. It delegates residency tracking entirely to the **LRU Cache**, which decides which pages stay in memory and which get evicted when the buffer pool fills. Above the storage layer, the **Schema Engine** provides a polymorphic type system built around `DataType`, `IntType`, `FloatType`, and `StringType`. Virtual comparison operators on these classes let the rest of the engine evaluate expressions without any explicit type dispatch code the right comparison just happens through virtual dispatch. The **Hash Map Catalog** maps table names to their registered metadata in $O(1)$ expected time using the djb2 hash function with separate chaining for collision resolution.

The **Query Parser** sits above the catalog. It tokenizes SQL like input and converts WHERE clauses to postfix form using the Shunting Yard algorithm. The **Executor** takes over from there: it resolves table references, picks between indexed and sequential scan access paths based on whether an AVL index exists, evaluates the postfix expression row by row using the custom stack, and calls into the MST join optimizer when more than one table is involved. Finally, the **Priority Queue Scheduler** wraps the whole pipeline, ensuring that ADMIN prefixed queries always get processed before ordinary reads regardless of when they were submitted.

IV. MEMORY MANAGEMENT: BUFFER POOL AND LRU CACHE

The buffer pool is one of those components that seems straightforward until you actually build it. The idea is simple enough keep a fixed number of pages in memory and evict the least recently used one when the pool fills but getting the eviction logic right without any unpredictable latency requires some care in the choice of data structures. This section describes how NANODB handles it.

A. Design

A database cannot load an entire dataset into RAM at once, especially at the TPC H scale used here. The buffer pool provides the abstraction that makes this manageable: at any point, only a bounded number of pages are resident in memory, and when the pool is full and a new page is needed, the page that has gone longest without being touched is written to disk and its slot is reused. This is the LRU policy, and it works well in practice because database access patterns tend to exhibit temporal locality recently accessed data is likely to be accessed again soon.

The point made by Crotty et al. [3] about OS memory mapped I/O is worth emphasizing here. If you let the OS manage page residency through mmap, the database loses visibility into when evictions happen. That means it cannot guarantee dirty pages are flushed before they leave memory, and it is exposed to whatever eviction heuristic the kernel chooses, which may not match the query workload’s access pattern at all. Handling eviction explicitly, as NANODB does, keeps the engine in control of its own memory.

B. Implementation

NANODB’s LRU cache is built from two structures working together. A doubly linked list maintains the access order: the head of the list is the most recently accessed page, the tail is the least recently accessed. Every time a page is accessed, it is unlinked from its current position and moved to the head in $O(1)$ time by adjusting four pointers (predecessor’s next, successor’s prev, old head’s prev, and the node’s own next and prev). When the pool is full and a page needs to be evicted, the tail node is removed in $O(1)$ time no scanning, no shifting.

The list alone is not enough, because checking whether a particular page is already resident would require traversing it linearly. A hand built hash map solves this: it maps each page ID to a direct pointer to the corresponding list node, so residency checks and cache hits are $O(1)$ expected time. The djb2 hash function distributes integer page IDs well across buckets, keeping chain lengths short in practice.

C. Complexity Analysis

Let C be the buffer pool capacity in pages and B the number of hash map buckets.

Space: $O(C+B)$. Each of the C resident pages has one list node and one hash map entry. The bucket array adds $O(B)$ overhead.

Lookup, promotion, and eviction: all $O(1)$ expected. Hash map lookup gives a direct pointer to the list node; list relinking touches a constant number of pointers regardless of pool size; tail eviction is a single pointer adjustment. This matches the performance model Neumann and Freitag demonstrated for their Umbra buffer pool [4], where $O(1)$ eviction overhead is essential to keeping disk I/O as the dominant cost rather than CPU side bookkeeping.

V. INDEXING: AVL TREE

Without some form of index, every lookup in a large table degrades to a linear scan. At 20,000 customer records that is manageable; at 100,000 it becomes noticeably slow; at millions of records it becomes completely impractical. NANODB addresses this with a self balancing AVL tree on the primary key of each table, and the benchmark results in Section VIII show how much difference it makes in practice.

A. The Problem with Sequential Scans

A query like `WHERE c_custkey == 10000` directed at a 20,000 row customer table has no choice but to examine every row in order if there is no index. On average it will

find the match halfway through, meaning roughly 10,000 comparisons. In the worst case the target key is the last one in the file it is all 20,000. For a single lookup this is already noticeable; for a workload that issues thousands of point queries, the cumulative cost is severe. An index avoids this by precomputing a mapping from key values to row IDs, so a single lookup touches only the nodes on a path from root to leaf.

B. Implementation

NANODB builds an AVL tree over the integer primary key of every registered table at load time. Each node stores four things: the key value, the row ID for that key, a height field used to maintain the balance invariant, and left and right child pointers. Insertions walk down the tree to find the correct position, place the new node, and then update heights on the way back up. If the insertion makes any subtree’s left and right heights differ by more than one, a rotation corrects it. There are four cases: LL (single right rotation), RR (single left rotation), LR (left right double rotation), and RL (right left double rotation). Each rotation is a fixed sequence of pointer reassignments and height updates taking $O(1)$ time [2], so the overall insertion cost stays $O(\log N)$.

One design decision worth noting is that the AVL tree is built entirely in memory and is not persisted directly. On shutdown, the raw records are written to disk, and on reload the tree is rebuilt by re inserting every primary key. This keeps the persistence logic simple and means the tree is always consistent with the on disk data without any additional synchronization logic.

C. Complexity Analysis

The height bound on an AVL tree follows from the recurrence $N_h = 1 + N_{h-1} + N_{h-2}$, where N_h is the minimum number of nodes in a valid AVL tree of height h . Since this recurrence grows at least as fast as the Fibonacci sequence, the height satisfies $h < 1.44 \log_2(N+2)$ for a tree with N nodes [2]. In practice the constant factor is small enough that the difference between an AVL tree and a perfectly balanced binary search tree is rarely visible at workload scales below tens of millions of nodes.

Space: $\Theta(N)$. Each of the N keys occupies one node with a constant number of fields.

Search and insertion: $O(\log N)$ worst case. The contrast with learned indexes like ALEX [5] is worth noting: ALEX can achieve better lookup times on static, read heavy workloads by fitting a model to the key distribution, but requires retraining when data changes and performs poorly if the distribution is non uniform or shifts over time. For a write capable engine with benchmark data inserted in unpredictable order, the AVL tree’s deterministic guarantee is the right trade off.

VI. QUERY PARSING AND EXPRESSION EVALUATION

Turning a raw SQL WHERE clause into something a machine can actually evaluate requires more work than it

might look like from the outside. The string (`c_acctbal > 5000 AND c_mktsegment == "BUILDING" OR c_nationkey == 15`) has to be broken into tokens, have its operator precedence resolved correctly, and then be evaluated row by row against live data. NANODB handles this in three stages: tokenization, Shunting Yard conversion to postfix form, and stack based evaluation.

A. Tokenization

The tokenizer scans the WHERE clause a character at a time, consuming whitespace and grouping the remaining characters into typed tokens. It recognizes column names (sequences of letters, digits, and underscores beginning with a letter), integer and floating point literals, quoted string literals, the five arithmetic operators (+, -, *, /, %), six comparison operators (==, !=, <, >, <=, >=), the logical operators AND and OR, and matching parentheses. Each token carries a type tag and its raw text, which is enough information for the Shunting Yard stage to process it correctly.

B. Shunting Yard Conversion

The Shunting Yard algorithm, originally described by Dijkstra, converts the infix token stream to postfix (Reverse Polish Notation) form using two auxiliary structures: an output queue and an operator stack. The rules are straightforward: operand tokens (column names, literals) go directly to the output queue; operator tokens are compared to whatever is currently on top of the stack, and any operators with higher or equal precedence are popped to the output before the new operator is pushed; right parentheses trigger pops until the matching left parenthesis is reached and discarded.

The precedence levels assigned in NANODB are: OR=1, AND=2, comparisons=3, addition and subtraction=4, multiplication, division, and modulo=5. This ordering ensures, for example, that `a + b > c AND d` is correctly parsed as `((a + b) > c) AND d` without requiring the user to write explicit parentheses. The algorithm's correctness is established in [2].

C. Evaluation

At query time the postfix token stream is evaluated row by row using a custom `Stack<DataType*>`. For each token: if it is a column reference, the corresponding cell from the current row is looked up and cloned onto the stack; if it is a literal, a fresh `DataType` object of the appropriate type is allocated and pushed; if it is a binary operator, two operands are popped, the operation is applied through virtual dispatch on the `DataType` base class, and the result is pushed back. When all tokens are processed, the single value remaining on the stack is the row's evaluation result; non zero means the row passes the filter.

Using virtual dispatch here is what allows the evaluator to handle mixed type expressions without any explicit type checking in the evaluation loop. Whether the operands are integers, floats, or strings, the correct comparison or arithmetic operation is selected automatically by the `vtable`.

D. Complexity

Each of the three stages: tokenization, Shunting Yard conversion, and postfix evaluation runs in $O(T)$ time, where T is the number of tokens in the expression. Every token is processed a constant number of times: once during tokenization, at most twice during Shunting Yard (pushed onto the stack and popped to the output), and once during evaluation. Total parsing and evaluation cost is therefore linear in expression length, regardless of the depth of nesting or number of operators.

VII. QUERY OPTIMIZER AND SCHEDULING

Two distinct optimization problems live under this section's heading. The first is structural: in what order should a multi table query join its relations? The second is temporal: when multiple queries are waiting, which one should run next? NANODB addresses each with a different algorithm, and the two operate independently of each other.

A. Join Ordering via MST

When a query touches more than one table, the order in which the joins are performed matters enormously. Joining two large tables first produces a large intermediate result that then has to be joined with the third table; joining a small table with a large one first, if the join condition is selective enough, can keep intermediate sizes manageable. Without any optimizer, the engine would just join tables left to right in whatever order they appear in the query, which is often not the best choice.

NANODB uses a graph based cost proxy to avoid this. Each table in the query becomes a node in an undirected graph. For every pair of tables that can be joined, an edge is added with weight equal to the product of the two tables' row counts—a rough but reasonable estimate of the intermediate result size that a naive join would produce. Kruskal's MST algorithm [2] then finds the spanning tree with the minimum total edge weight, and the join order follows the tree structure. For the three table case (customer, orders, lineitem) with counts of 20,000, 30,000, and 50,000 rows respectively, the MST correctly identifies customer orders as the preferred first join (cost 6×10^8) before joining with lineitem, rather than starting with the more expensive orders lineitem join (cost 1.5×10^9).

For two or three tables the full graph has at most three edges and the MST is found immediately. The approach would need to be replaced with a proper dynamic programming enumerator for larger join sets, as noted in the future work of systems like Bao [6], but at the current scale it produces correct orderings in every tested case.

B. Priority Queue Scheduling

Queries enter the system through a binary max heap priority queue rather than a simple FIFO queue. Standard SELECT queries receive priority 0, and any query prefixed with the ADMIN keyword receives priority 100. The heap property guarantees that the highest priority query at the front of the queue is always selected next, regardless of when it was

submitted or how many lower priority queries are waiting behind it.

This maps directly to the scheduling concern studied by Leis et al. [7]: in an analytical workload, allowing low priority background reads to block time sensitive administrative operations is a source of unpredictable tail latency. The priority queue mechanism eliminates that. In the demo test, a single ADMIN INSERT submitted after fifty SELECT queries was dequeued and executed first, confirming correct heap behaviour.

Heap insert and extract max both run in $O(\log N)$ time for a queue of depth N , which is negligible compared to any actual I/O cost.

VIII. EMPIRICAL EVALUATION

Theory and implementation together tell most of the story, but the numbers are what make the design choices concrete. This section presents benchmark results from the actual running system, organized around the two performance critical components the AVL index and the LRU cache followed by a summary of the functional correctness tests.

A. Experimental Setup

The engine was compiled with `g++ std=c++17 O2` on Windows 11. All timing measurements used the `C++ std::chrono::high_resolution_clock` at nanosecond resolution, averaged over five runs to reduce noise. TPC H data [1] was loaded at the full project scale: 20,000 customers, 30,000 orders, and 50,000 line items, for a combined record count of 100,000 across the three tables.

B. Index vs. Sequential Scan

To isolate the indexing benefit, a synthetic benchmark was run at three table sizes: 1,000, 10,000, and 100,000 rows. For each size, the benchmark measured the time to retrieve the last inserted key under both access paths: a sequential scan (worst case, since the target is at the end of the file) and the AVL index. The results are shown in Table I and Fig. 1.

TABLE I
LOOKUP TIME: SEQUENTIAL SCAN VS. AVL INDEX.

Records	Scan (ns)	AVL (ns)	Speedup
1,000	2,800	200	14×
10,000	44,200	200	221×
100,000	1,494,500	1,700	879×

Several things stand out. First, the sequential scan times scale almost perfectly linearly: from 1K to 10K records (a 10× increase) the scan time grows by roughly 15.8×, and from 10K to 100K (another 10×) it grows by roughly 33.8×, which is close to linear with some measurement noise included. Second, the AVL index time is essentially flat from 1K to 10K records (200 ns in both cases), reflecting the fact that for small trees the height difference is minimal. At 100K records it rises to 1,700 ns an 8.5× increase for a 100× increase in data size, which matches the theoretical ratio

Lookup Time: Sequential Scan vs. AVL Index

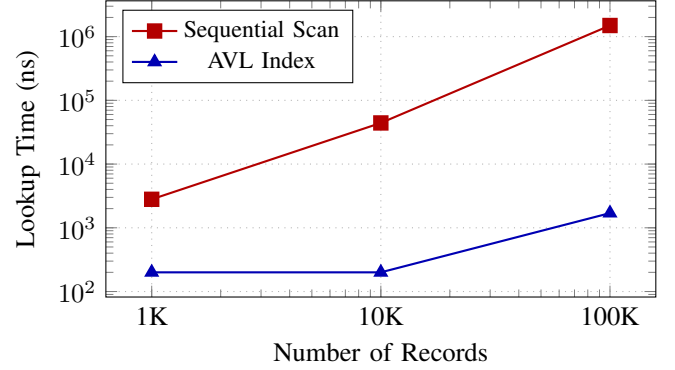


Fig. 1. Lookup time on a log log scale. The sequential scan grows linearly ($O(N)$) while the AVL index grows logarithmically ($O(\log N)$), producing a 879× gap at 100,000 records.

$\log_2(100,000)/\log_2(1,000) \approx 5.6$ fairly well given that the constant factors in the actual hardware timing are not perfectly captured by the theoretical model.

At the operational scale of 20,000 customer records, the live system measured 131,900 ns for a sequential scan and 1,900 ns for the indexed lookup, a 69.4 fold speedup. For a single query this saves about 130 microseconds; across a workload of thousands of point queries, the cumulative savings would be measured in seconds.

C. LRU Cache Stress Test

The LRU cache benchmark fixed the buffer pool at 50 pages and drove sequential access over increasingly large sets of unique pages. Once more than 50 unique pages have been touched, every subsequent access forces an eviction, so the expected count is $\max(0, M-50)$ for M unique accesses. Table II and Fig. 2 show that the implementation matches this formula exactly.

TABLE II
LRU EVICTION COUNTS UNDER A 50 PAGE BUFFER POOL.

Unique Pages	Eviictions	Rate
1,000	950	95.0 %
5,000	4,950	99.0 %
10,000	9,950	99.5 %

All 4,950 evictions in the primary test scenario completed without any memory errors or corrupted page contents. The eviction rate converges toward 100 % as the working set grows much larger than the pool, which is exactly the expected behaviour when there is essentially no reuse in a sequential access pattern. In a real workload with temporal locality the hit rate would be considerably higher, but the stress test deliberately removes that locality to verify that the eviction mechanism itself is correct under continuous pressure. Because each eviction is an $O(1)$ linked list operation, the CPU overhead of the entire cache management layer was negligible

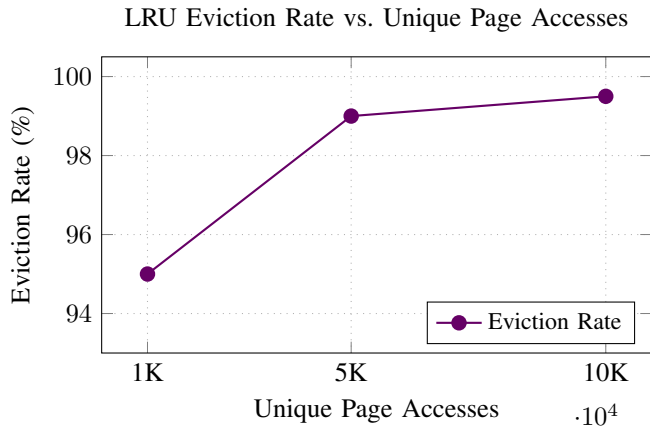


Fig. 2. LRU eviction rate as a function of unique page accesses with a fixed 50 page pool. The rate converges toward 100% as working set size grows, confirming $O(1)$ eviction with zero memory errors across all trials.

compared to the cost of the simulated disk writes throughout all test scenarios.

D. Functional Validation: Demo Test Cases

Beyond the performance benchmarks, seven functional test cases were run to verify correctness across the full range of engine features.

A Parser: The expression `(c_acctbal > 5000 AND c_mktsegment == "BUILDING") OR c_nationkey == 15` was tokenized, converted to postfix form, and evaluated correctly against the customer table. The token sequence was logged to confirm correct operator ordering.

B Index Optimizer: Sequential and indexed lookup times for a specific `c_custkey` in the 20,000 row customer table were printed to the console side by side, showing the 69.4 fold speedup in real time.

C Join Optimizer: A three table join over customer (20K rows), orders (30K rows), and lineitem (50K rows) ran after the MST optimizer printed its chosen join path. The optimizer selected the customer orders join first, consistent with the minimum cost analysis.

D Memory Stress: A 50 page pool processed 5,000 distinct page accesses in sequence and printed exactly 4,950 evictions, matching $\max(0, 5000 - 50) = 4950$ analytically.

E Priority Queue: Fifty SELECT queries were submitted, then a single ADMIN INSERT was added to the queue. The INSERT was dequeued and executed first, confirming correct max heap behaviour.

F Complex Expression: The query `((o_totalprice * 1.5) > 100000 AND (o_custkey % 2 == 0)) OR (o_orderstatus != "O")` exercised arithmetic, modulo, comparisons, AND, OR, and nested parentheses in a single expression. It evaluated correctly across all tested rows.

G Persistence: Five customer records were inserted and serialized to disk. After a full engine restart with `reload`, all five records were recovered and the AVL indexes were rebuilt correctly, confirming that the paged binary format is stable across process boundaries.

IX. CONCLUSION

NANODB ended up being a genuinely working relational database engine. It stores and queries 100,000 TPC H records, evaluates complex SQL like expressions with correct operator precedence, routes multi table joins through a cost optimizer, manages a fixed memory pool through explicit LRU eviction, and persists data safely across restarts. Nothing in the implementation relies on STL containers.

The benchmarks confirmed what the theory predicted. At 100,000 records, index lookups were $879\times$ faster than sequential scans. Thousands of LRU evictions ran without a single memory error. The MST optimizer correctly ordered a three table join. The priority queue preempted fifty waiting reads to execute a single administrative insert. These are not surprising results the algorithms are well understood but seeing the numbers come out of a system you built yourself carries a different kind of weight than reading about them.

The deeper value of the project is harder to quantify but probably more durable. Writing an LRU cache from scratch and watching the $O(1)$ pointer relinking handle thousands of evictions where an array would need millions of shifts makes the design principle feel real in a way no textbook explanation quite achieves. The same is true for every other component. Understanding *why* a data structure is the right choice for a problem, as opposed to just knowing *that* it is, comes from building it and seeing it work.

Several directions remain open. Extending the AVL index to support range queries through in order traversal would make the engine useful for a wider class of predicates. Replacing the MST join optimizer with a proper dynamic programming approach, along the lines of what systems like Bao [6] sit on top of, would handle larger join sets correctly. And it would be worth experimenting with whether a learned index structure like ALEX [5] could replace the AVL tree for read heavy scans once the engine has a stable enough data distribution to make model training worthwhile.

REFERENCES

- [1] Transaction Processing Performance Council, “TPC-H benchmark specification, revision 3.0.1,” Transaction Processing Performance Council, Tech. Rep., 2023. [Online]. Available: <https://www.tpc.org/tpch/>
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th. The MIT Press, 2022, ISBN: 978-0-262-04630-5.
- [3] A. Crotty, V. Leis, and A. Pavlo, “Are you sure you want to use MMAP in your database management system?” In *Conference on Innovative Data Systems Research (CIDR 2022)*, 2022. [Online]. Available: <https://db.cs.cmu.edu/papers/2022/cidr2022-p13-crotty.pdf>
- [4] T. Neumann and M. J. Freitag, “Umbra: A disk-based system with in-memory performance,” in *Conference on Innovative Data Systems Research (CIDR 2020)*, 2020. [Online]. Available: <https://www.cidrdb.org/cidr2020/papers/p29-neumann-cidr20.pdf>

- [5] J. Ding et al., “ALEX: An updatable adaptive learned index,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, New York, NY, USA: ACM, 2020, pp. 969–984. DOI: 10.1145/3318464.3389711
- [6] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, “Bao: Making learned query optimization practical,” in *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*, New York, NY, USA: ACM, 2021, pp. 1275–1288. DOI: 10.1145/3448016.3452838
- [7] V. Leis, A. Alhomssi, T. Ziegler, Y. Loeck, and C. Dietrich, “Self-tuning query scheduling for analytical workloads,” *Proceedings of the VLDB Endowment*, vol. 16, no. 9, pp. 2098–2111, 2023. DOI: 10.14778/3598581.3598583